MANIPAL UNIVERSITY JAIPUR
Department of Computer Science and Engineering

CSE3132 | Software Engineering Laboratory

PROJECT REPORT

Smart Attendance System using Face-Recognition Scanner

Software Requirements Specification, UML design, estimation, testing and results

Submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering

Submitted by

Bhoomi Saraf | Kanakpreet Kaur | Apoorva Kala

Course Code: CSE3132 Academic Year: 2026-2027

Submission date: 06 November 2026

Abstract

Manual roll-call consumes teaching time, invites clerical errors and allows proxy attendance. The Smart Attendance System automates class attendance with a camera: a teacher starts a session for the course section they teach, the browser sends webcam frames to a Flask server, the server detects and recognises the face against the roster of that section only, and a present record is stored once per student per session with a timestamp and a confidence value.

The system has two roles. Administrators own master data: they register students (each belongs to a section), create courses, assign a teacher to every course section, edit rosters and capture reference faces. Faculty see only the sections they teach, run attendance sessions, correct records manually, and view section-aware reports with a 75% eligibility flag and CSV export.

This report is written in Software Requirements Specification (SRS) format. It contains the requirements, the UML models (use case, class, sequence, object, package and activity diagrams), the effort estimation, the testing techniques, the test cases, and the measured results. All results quoted here come from the automated test suite (94 tests, all passing) and from recognition experiments run on the enrolled demo faces; Section 9 states exactly what was and was not measured.

Keywords: face recognition, attendance management, SRS, UML, SFace, role-based access, section-wise rosters, software testing.

Contents

1 Introduction
2 Overall Description
3 Specific Requirements
4 System Design and UML Diagrams
5 Implementation Summary
6 Effort and Schedule Estimation
7 Testing Strategy and Techniques
8 Test Cases
9 Results and Evaluation
10 Limitations and Future Work
11 Conclusion
References
Appendix A Running and demonstrating the system

1 Introduction

1.1 Purpose

This document specifies the requirements, design and verification of the Smart Attendance System. It is intended for the course evaluator, for the project team, and for anyone who will maintain or extend the system.

1.2 Problem statement and motivation

Calling out names or passing a register takes several minutes per lecture, is error-prone and does not stop one student answering for another. Card or QR schemes can be shared. The project replaces this with automatic face-based identification tied to a specific course, section and class session, with central storage and report generation.

1.3 Scope

In scope (implemented)	Out of scope / future work
Administrator functions: student registration with section, course creation, teacher-to-section assignment, roster management, face capture	Liveness / anti-spoofing against photos and video replay
Faculty login, session start for an assigned section, live webcam scanning, manual override, session close	Integration with the institution ERP
Face detection, quality checking, alignment and matching restricted to the section roster	Dedicated mobile scanning application
Duplicate prevention, section and enrolment validation, attendance percentage with 75% flag	E-mail / SMS low-attendance alerts (the flag is shown in reports and CSV)
Section-aware reports, per-session present/absent report, CSV export	Student login and self-service attendance view
Role-based access control, CSRF protection, SQLite persistence	Audit trail of manual corrections

1.4 Definitions and abbreviations

Term	Meaning
Section	A subdivision of a batch (A, B, C). Every student belongs to one section.
Course section	One section of one course, taught by exactly one faculty member (table course_sections).
Roster	Students enrolled in the course who belong to the session's section.
Embedding	A 128-number vector describing a face; produced by the SFace network.
Cosine similarity	Similarity score between two embeddings (1 = identical direction). Matching threshold is 0.50.
YuNet / SFace	OpenCV Zoo ONNX models for face detection (with landmarks) and face recognition.
FAR / FRR	False acceptance rate / false rejection rate.
RBAC / CSRF	Role-based access control / cross-site request forgery.

1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications; ISO/IEC/IEEE 29148:2018.
- Sommerville, I., Software Engineering, 10th ed., Pearson. Pressman, R. S., Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill.
- OMG, Unified Modeling Language Specification, version 2.5.

- Zhong, Y. et al., SFace: Sigmoid-Constrained Hypersphere Loss for Robust Face Recognition, IEEE TIP, 2021; OpenCV Zoo: YuNet and SFace models.
- Project synopsis and Labs 2, 3, 5 (use case, class, package diagrams) of CSE3132.

1.6 Document overview

Section 2 describes the product in general terms; Section 3 lists the specific requirements; Section 4 presents the design with all UML diagrams; Section 5 summarises the implementation; Section 6 gives the estimation; Sections 7 to 9 cover testing and results; Sections 10 and 11 close the report.

2 Overall Description

2.1 Product perspective

The system is a self-contained web application. The browser (Bootstrap interface, WebRTC camera) talks to a Flask server. The server applies authentication, CSRF and role checks, delegates business rules to the AttendanceService, and uses the FaceRecognitionService for detection and matching. Data is stored in a SQLite database; face embeddings are stored in a compressed NumPy file.

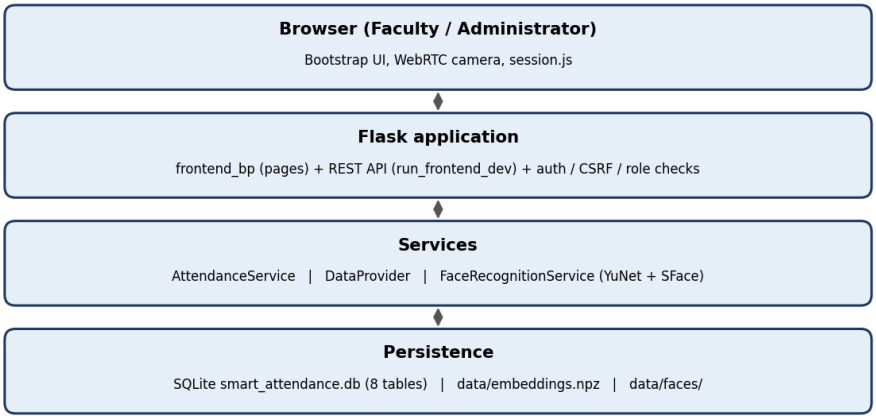


Figure 1. Layered architecture of the Smart Attendance System

2.2 Product functions

- Master-data management by the administrator: students with sections, courses, teacher-per-section assignment, rosters.
- Face enrolment from the webcam (5 frames) or from a photo, with a quality gate.
- Section-restricted attendance sessions with live recognition, agreement across scans, duplicate and wrong-section rejection.
- Manual override for edge cases, session close, per-session present/absent report.
- Course- and section-wise attendance percentages, 75% eligibility flag, role-scoped CSV export.

2.3 User classes

User class	Capabilities	Restrictions
Administrator	Register students, create courses, assign teachers to sections, edit rosters, capture faces, view all reports	Does not start attendance sessions
Faculty	Start sessions for assigned sections, scan, mark manually, close sessions, view own reports and CSV	Sees only own course sections and their students; cannot change master data
Student	Is the subject of recognition; no login in this version	No access to the portal (decision resolves the use-case/sequence-diagram mismatch noted in the synopsis)

2.4 Operating environment

- Windows/Linux/macOS with Python 3.10+, Flask, Flask-WTF, OpenCV 4.8+, NumPy, SQLite3.
- A webcam and a modern browser with camera permission. Recognition runs on CPU.

2.5 Design and implementation constraints

- Face recognition must run locally (no cloud service) and must not store raw biometric images in the database; only embeddings are stored (reference photos are kept in a git-ignored folder so embeddings can be rebuilt if the model changes).
- Prototype scope: SQLite, Flask development server, no HTTPS setup, no liveness detection.

2.6 Assumptions and dependencies

- Each student has a registered profile and an enrolled face before recognition is attempted.
- The camera has reasonable lighting and the student faces the camera; the system reports when this is not the case instead of guessing.
- The YuNet and SFace ONNX models are downloaded once with fetch_models.py.

3 Specific Requirements

3.1 External interface requirements

3.1.1 User interface

Browser-based interface with a login page, a dashboard (administrator console or faculty dashboard), a students page, a courses page, a live session page with the camera viewport and recognition HUD, and a reports page. Dark navy theme, Bootstrap 5, responsive layout.

3.1.2 Hardware and software interfaces

- Webcam via browser getUserMedia (WebRTC). Frames are posted as JPEG data URIs.
- SQLite database file smart_attendance.db; embeddings in data/embeddings.npz; ONNX models in models_cv/.

3.1.3 Principal HTTP interfaces

Method and path	Role	Purpose
POST /login, GET /logout	Any	Authenticate / end session
POST /actions/students/add	Admin	Register a student with a section
POST /actions/courses/create	Admin	Create a course and its sections
POST /actions/courses/{id}/sections	Admin	Assign or change the teacher of a course section
POST /api/courses/{id}/enroll	Admin	Replace a course roster
POST /api/students/{id}/register-face	Admin	Enrol face from one or several frames
POST /actions/sessions/start	Faculty	Start a session for an assigned course section
POST /api/sessions/{id}/recognize	Faculty / camera key	Recognise a frame and mark attendance
POST /api/sessions/{id}/mark-manual	Faculty	Manual present mark
POST /api/sessions/{id}/end	Faculty	Close the session
GET /api/reports/export-csv	Faculty / Admin	CSV scoped by role, course and section

3.2 Functional requirements

Every functional requirement below is implemented and is verified by at least one automated test (Section 8).

Table 1. Functional requirements and their verification

ID	Requirement	Status	Verified by
FR-01	Administrator registers and updates student records, including the student's section.	Implemented	test_new_student_is_enrolled_in_their_sections_courses
FR-02	Faculty records exist and are linked to login accounts.	Implemented	test_2_faculty_assignment_and_course_attributes
FR-03	Administrator creates courses and assigns a teacher to each course section; rosters are maintained per course.	Implemented	test_admin_creates_course_with_sections_and_enrols_them; test_admin_reassigns_section_teacher
FR-04	Administrator enrols a student's face; the template is linked to the student record.	Implemented	test_4_face_encoding_persistence_and_failure_honesty
FR-05	Faculty authenticate before starting a session.	Implemented	test_login_with_seeded_faculty; test_invalid_password_rejected
FR-06	Faculty select a course and a section they teach and start a session.	Implemented	test_session_roster_is_the_chosen_section_only; test_faculty_cannot_start_unassigned_section
FR-07	The system captures the live camera feed and detects faces during an active session.	Implemented	test_live_session_page_config_is_valid_javascript; test_valid_recognised_enrolled_student_marked_once

ID	Requirement	Status	Verified by
FR-08	Detected faces are compared with the templates of the session roster only.	Implemented	test_match_is_restricted_to_the_allowed_roster
FR-09	A recognised student is recorded with student id, timestamp and confidence.	Implemented	test_valid_recognised_enrolled_student_marked_once
FR-10	Duplicate attendance for the same student in the same session is prevented.	Implemented	test_repeated_request_does_not_create_second_attendance_record
FR-11	Faculty can view attendance and correct it manually.	Implemented	test_manual_override; test_manual_mark_rejects_other_section_and_duplicates
FR-12	Attendance percentages and course/section/student-wise reports are produced.	Implemented	test_report_percentages_are_per_section; test_session_report_lists_present_and_absent
FR-13	Reports can be restricted to a course and a section.	Implemented	test_report_percentages_are_per_section; test_csv_export_is_scoped_to_the_teacher
FR-14	Students below 75% are flagged as low attendance.	Implemented (flag in reports and CSV; no e-mail alert)	test_attendance_reports_and_percentages
FR-15	Profiles, templates, courses and attendance persist in a database / file store and survive restart.	Implemented	test_6_true_restart_persistence
FR-16	Attendance data can be exported to CSV.	Implemented	test_csv_export; test_7_csv_export_multi_course_and_edge_cases
FR-17	Face quality problems (too small, blurry, dark, no face, several faces) are reported to the user.	Implemented	test_dark_frame_is_reported_as_low_quality_not_guessed; test_two_faces_in_one_frame_are_rejected
FR-18	Master-data actions are restricted to administrators; faculty see only their own sections.	Implemented	test_faculty_cannot_create_course_or_student; test_faculty_only_sees_own_courses_and_students

3.3 Non-functional requirements

Table 2. Non-functional requirements

ID	Attribute	Requirement	How it is met / measured
NFR-01	Usability	Starting a session takes few steps; failures are explained.	Course then section dialog; HUD messages such as "Move closer", "Add more light", "Wrong section".
NFR-02	Performance	Recognition must feel immediate.	Measured 63 ms mean (95th percentile 83 ms) per frame on CPU, excluding capture and network.
NFR-03	Reliability	No duplicate present record per student per session.	UNIQUE(session_id, student_id) plus application check; tested.
NFR-04	Accuracy	Threshold balances false accept and false reject.	Threshold 0.50, margin 0.08, agreement of two scans; evaluated in Section 9.2.
NFR-05	Security	Role-controlled access; credentials and biometrics protected.	Hashed passwords (scrypt), CSRF tokens, RBAC decorators, API key for camera kiosks; 21 security tests.
NFR-06	Privacy	Biometric data only for attendance.	Only embeddings in the registry; photos in a git-ignored folder; admin-only enrolment.
NFR-07	Maintainability	Separate user, course, attendance, recognition and persistence concerns.	Modules: models, attendance_service, data_provider, face_recognition_service, frontend_bp.
NFR-08	Availability	No false success on camera or database failure.	test_database_write_failure_not_reported_as_success.
NFR-09	Auditability	Manual corrections identifiable.	marking_method and verified_by stored per record; full audit trail left as future work.
NFR-10	Portability	Runs on common OS with a webcam.	Pure Python, ONNX models, SQLite; no GPU needed.

3.4 Logical database requirements

Eight tables: users, faculty, students, courses, course_sections, enrollments, class_sessions and attendance_records. Foreign keys are enforced. The two uniqueness rules that matter most are UNIQUE(course_id, student_id) on enrollments and UNIQUE(session_id, student_id) on attendance_records.

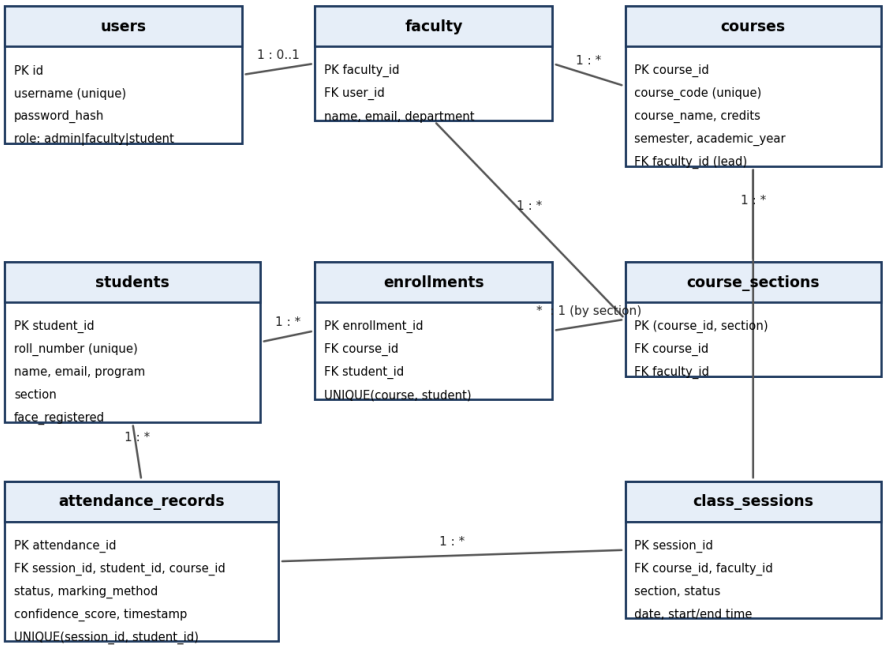


Figure 2. Entity-relationship diagram of the SQLite schema

4 System Design and UML Diagrams

The six diagrams below are complementary views of the implemented system. They were redrawn for this report to match the final design, in particular the course-section model and the role split, and they supersede the earlier lab diagrams where those differ. The student-login question raised in the synopsis (sequence diagram shows student login, use-case case study says it is not needed) is resolved in favour of no student login: students are identified by their face only.

4.1 Use case diagram

Three actors interact with the system boundary: the Administrator, the Faculty member and the Camera / face-recognition component, which feeds recognition results into the attendance use case.

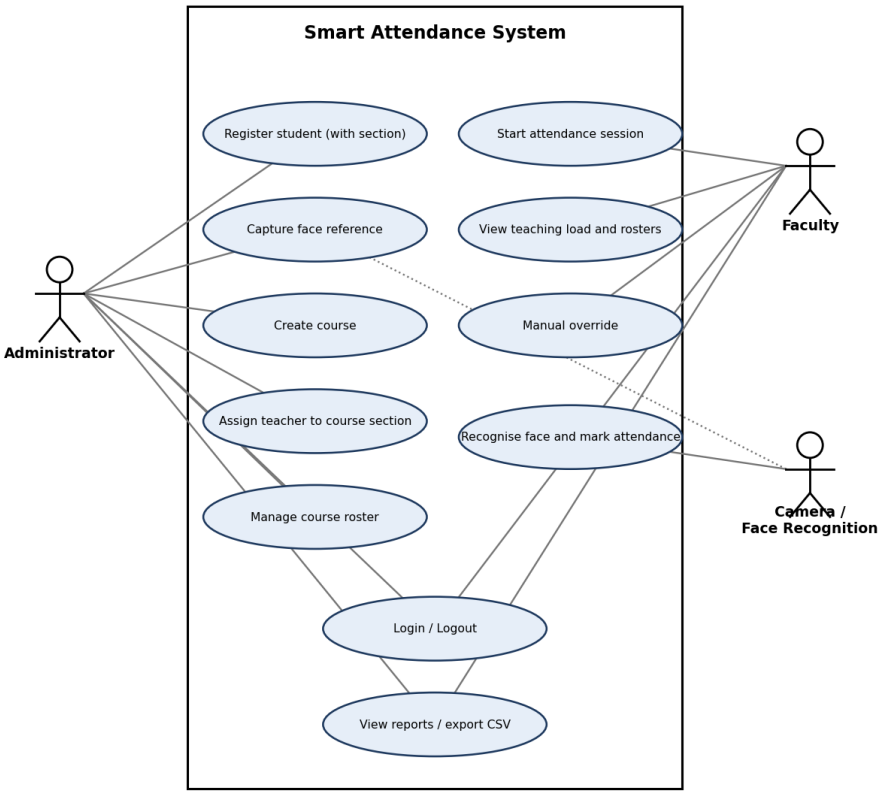


Figure 3. Use case diagram

Table 3. Use case descriptions (main flows)

Use case	Actor	Main flow	Alternative / exception
Register student	Admin	Enter roll number, name, e-mail, section; student is saved and enrolled in every course taught to that section.	Duplicate roll or e-mail: existing record returned.
Capture face	Admin	Open camera, 5 frames captured 0.4 s apart, quality-checked, embedded, stored; flag set to Enrolled.	No/multiple faces or poor quality: message, nothing stored.
Assign teacher	Admin	Choose course and section, select faculty; section students are enrolled.	Blank teacher removes the section.
Start session	Faculty	Choose a course then a section from own assignments; session becomes Active.	Unassigned section: refused (403-style message).
Mark attendance	Faculty / Camera	Frame is sent every 1.5 s; face matched on the roster; confirmed by agreement; record saved.	Unknown, ambiguous, low quality, duplicate, wrong section: no record, reason shown.
Manual override	Faculty	Select a roster student and mark present.	Duplicate or wrong section rejected.
View reports	Faculty / Admin	Filter by course and section; see percentages, 75% flag, per-session present/absent; export CSV.	Faculty see only own sections.

4.2 Class diagram

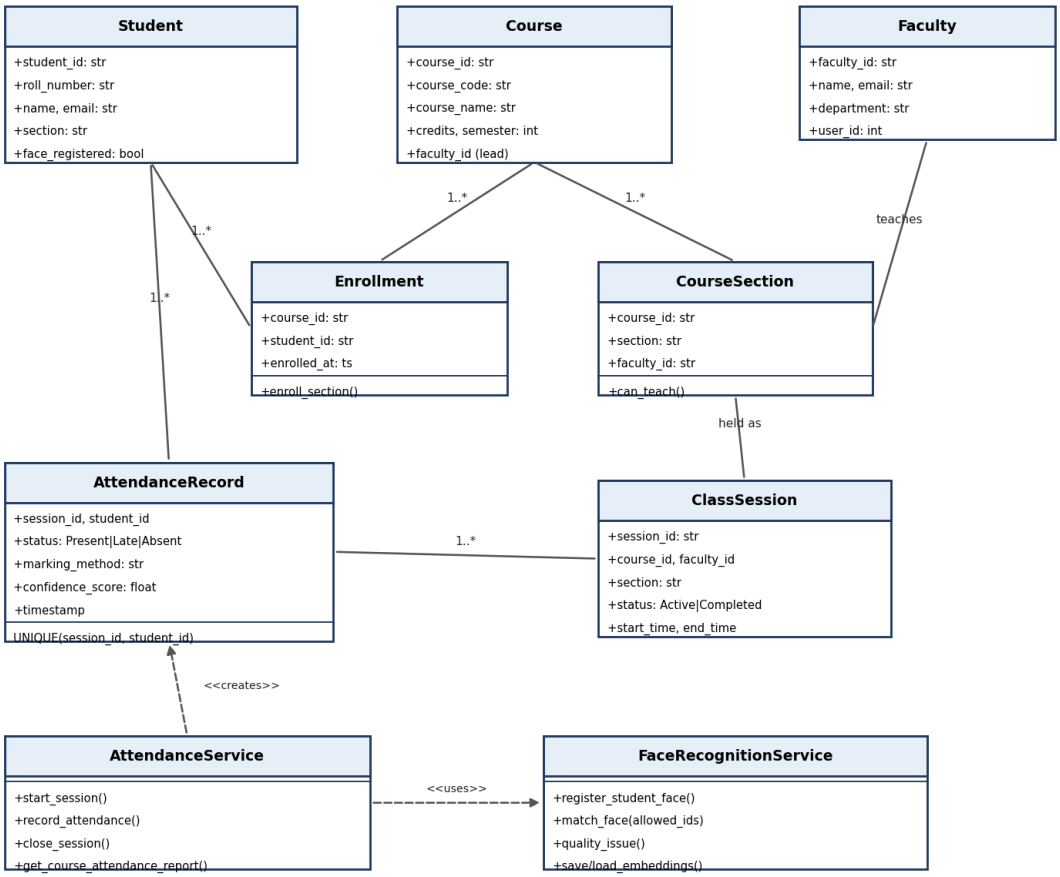


Figure 4. Class diagram of the domain and service classes

The class set that the synopsis said had to be reconciled before implementation is now one model: Student, Faculty, Course, CourseSection, Enrollment, ClassSession and AttendanceRecord as domain classes; AttendanceService and FaceRecognitionService as service classes. AcademicAdministrator is represented by a user with role admin.

4.3 Sequence diagram

The diagram follows one complete class: session start, repeated recognition with the agreement rule, the validation inside AttendanceService, and session close.

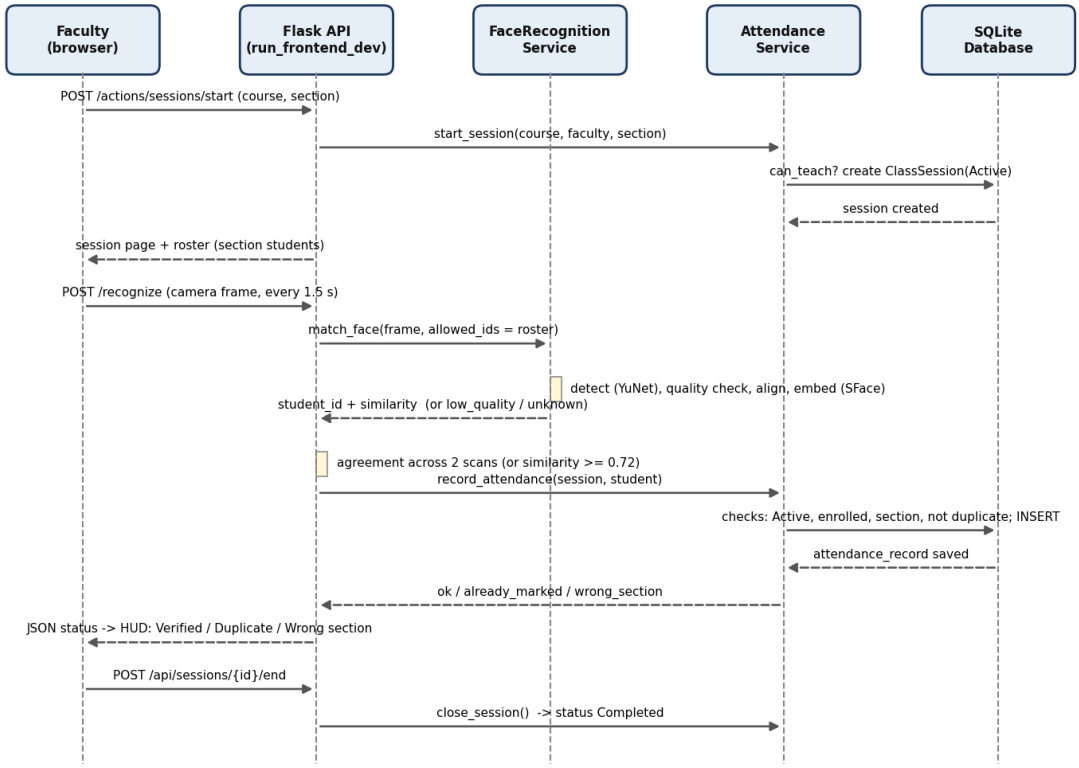


Figure 5. Sequence diagram: start session, recognise and mark attendance, close session

4.4 Object diagram

A snapshot of the demo database while a section A session is running. The values are the real demo records.

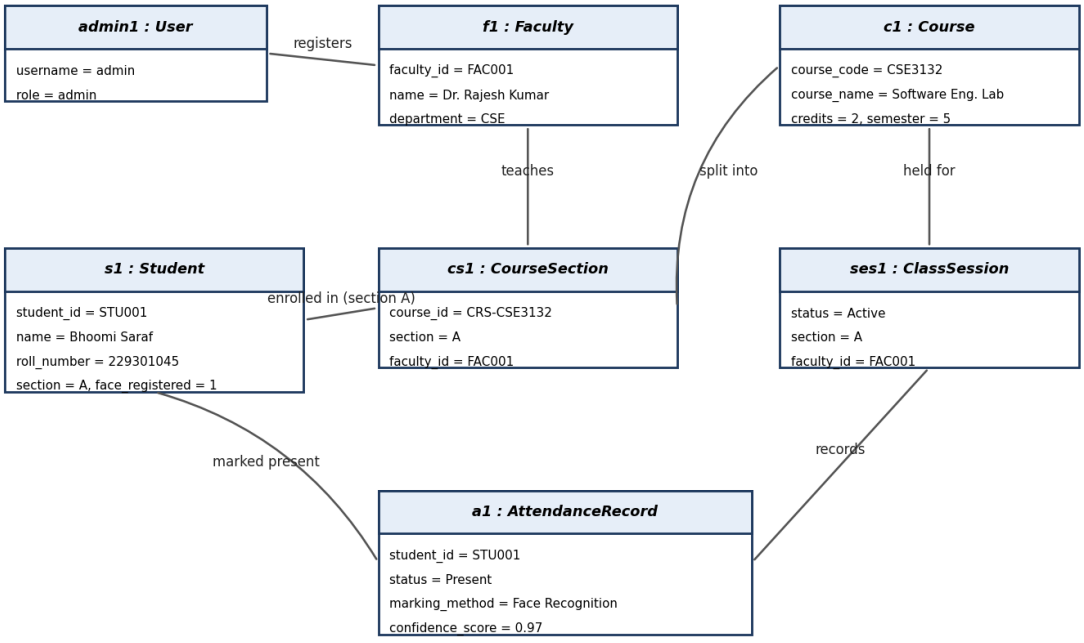


Figure 6. Object diagram

4.5 Package diagram

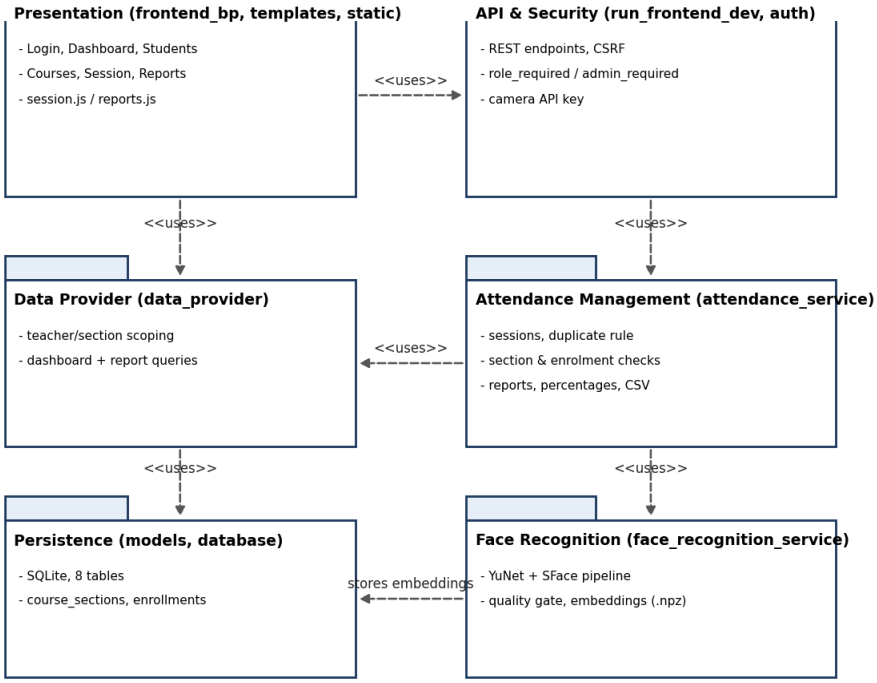


Figure 7. Package diagram of the six modules

4.6 Activity diagram

Five swimlanes show the whole workflow from administrator set-up to session close and reporting, including the decision points for credentials, face usability, roster match, agreement of scans and duplicate or wrong-section checks.

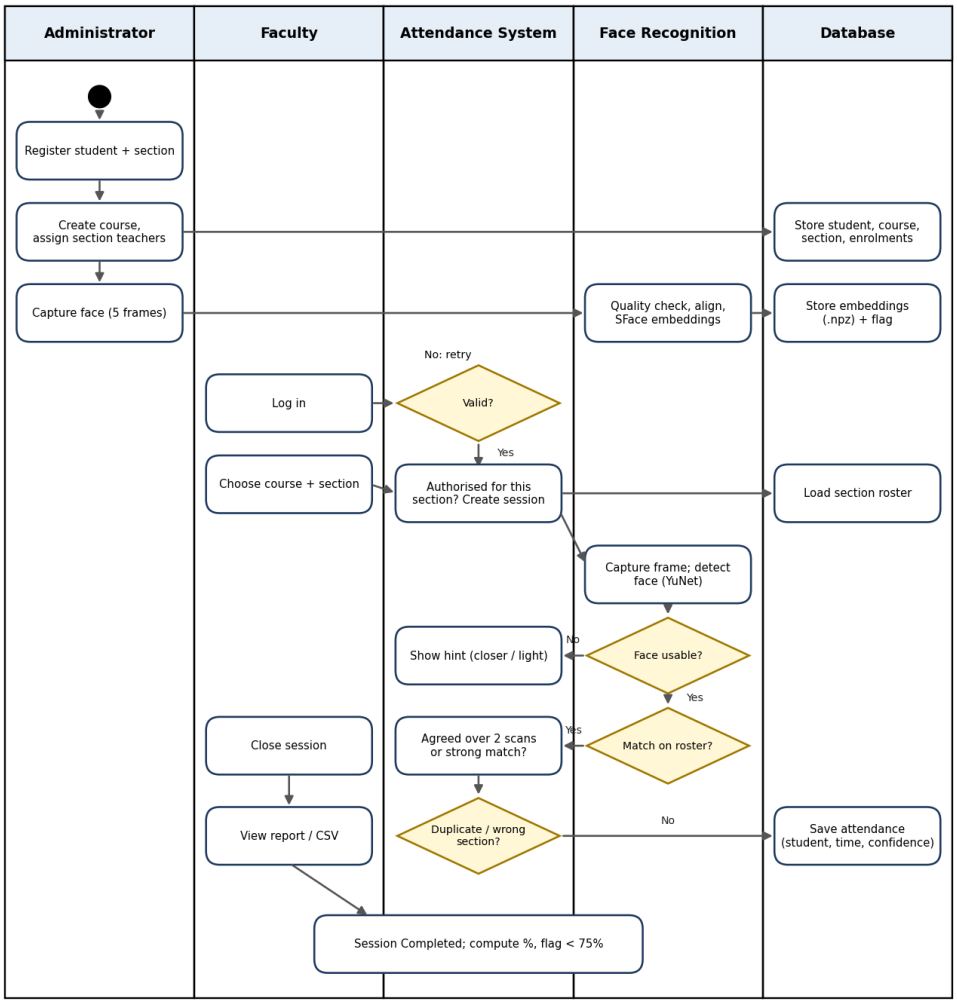


Figure 8. Activity diagram with swimlanes and decision branches

4.7 Face recognition pipeline design

The recognition engine was redesigned after the beta test (Section 9.3). It now has the following stages.

Table 4. Face recognition pipeline

Stage	Design	Reason
1. Detect	YuNet detector with 5 landmarks; the largest face wins, faces under 35% of its area (posters, background) are ignored.	Robust to background and small faces.
2. Quality gate	Minimum face size, detector confidence, sharpness (Laplacian variance), brightness range.	Poor frames give a hint ("move closer", "add light") instead of a random match.
3. Align and embed	Landmark alignment to 112x112, SFace 128-d embedding. Probe = average of the face, its mirror and a lighting-normalised copy.	Identity is judged on the aligned face only, so background and head tilt do not matter.
4. Enrol	Five variants per photo (original, mirror, CLAHE, brighter, dimmer); webcam enrolment takes 5 frames.	One capture already covers common lighting changes.
5. Match	Cosine similarity against the roster of the session only; threshold 0.50; best must beat the runner-up by 0.08.	Restricts the search space and refuses look-alike ties.
6. Confirm	A match is accepted from one scan only at similarity >= 0.72; otherwise the same student must win two consecutive scans.	One lucky frame cannot mark attendance.
7. Validate	AttendanceService checks Active session, enrolment, section and duplicates before saving.	Business rules stay in one place.

4.8 Security design

- Passwords are stored as scrypt hashes; login failures give no hint which field was wrong.
- Session-authenticated POST requests require a CSRF token; camera kiosks use an API key and are limited to the recognition endpoint.
- Decorators enforce the roles: faculty_required, admin_required; faculty are also checked against the session owner and the course section.
- Embeddings produced by a different model are rejected on load (dimension check) so stale data cannot silently mis-match.

5 Implementation Summary

The prototype is written in Python (Flask) with a Bootstrap front end. Application source is about 4,100 lines of Python and 3,050 lines of templates, JavaScript and CSS; the automated tests add about 2,600 lines.

Table 5. Modules

Module	Responsibility
run_frontend_dev.py	Flask app, REST API (recognise, mark-manual, end session, enrol, face registration, CSV), CSRF and error handlers
frontend_bp.py	Page routes and role-scoped views (dashboard, students, courses, session, reports)
auth.py, config.py	Authentication, hashing, role decorators; configuration and production safeguards
attendance_service.py	Business rules: authorised start, active session, enrolment, section, duplicate, reports, CSV
data_provider.py	Teacher-scoped queries for dashboard, students, courses, reports
models.py, database.py	SQLite access, schema, CourseSectionModel, EnrollmentModel and others
face_recognition_service.py	YuNet + SFace pipeline, quality gate, embeddings store, legacy fallback used by synthetic-frame tests
seed_demo.py, seed/sample_data.json, enroll_faces.py, fetch_models.py	Demo data (11 faculty, 10 courses, 27 course-section assignments, 30 students), face enrolment from photos, model download
templates/, static/	Pages, live scanner (session.js), reports (reports.js), theme

Demo data: 11 faculty, 10 courses (core, electives and labs), 27 course-section assignments and 30 students in sections A, B and C. Students for whom a photo was supplied are enrolled with the face pipeline; the rest are sample records without a face.

6 Effort and Schedule Estimation

Three views are given: the bottom-up plan made at the start, a revised bottom-up estimate after the scope grew during the beta test, and a model-based estimate from the measured size. The hour figures are planning values; the team did not keep time sheets, so they are not presented as measured effort.

6.1 Bottom-up estimate

Table 6. Work breakdown (person-hours)

Work package	Initial plan	Revised estimate
Requirements clarification and SRS	6	8
UML and data model refinement	8	10
UI and authentication workflow	10	22
User, course, section management and persistence	12	20
Face capture and recognition integration	16	30
Attendance session and record logic	12	16
Reports, percentages and CSV	8	10
Integration, testing and fixes	12	24
Documentation and demonstration	6	10
Total	90 h (11.25 person-days)	150 h (18.75 person-days)

The largest growth is in the recognition work and in the UI: the first recognition engine used a general image classifier and had to be replaced after the beta test, and the section-wise role model added screens and rules that were not in the initial plan.

6.2 Size measurement and COCOMO

Measure	Value
Application source (Python)	4,078 lines
Templates, JavaScript, CSS	3,049 lines
Total application size	7.1 KLOC
Automated tests	94 tests, about 2,600 lines
Basic COCOMO (organic): effort = 2.4 x KLOC^1.05	2.4 x 7.1^1.05 = 18.8 person-months
Basic COCOMO: schedule = 2.5 x effort^0.38	2.5 x 18.8^0.38 = 7.7 months

The COCOMO figure assumes professional productivity and full-time staff; it is far above the hours a three-person student team spent, because much of the code is template markup and tests and because libraries do the heavy work (OpenCV models, Flask, Bootstrap). It is quoted as a model-based upper reference, not as a prediction for this team. The bottom-up estimate is the more realistic of the two.

6.3 Milestones

Milestone	Deliverable / exit condition	Status
M1 Requirements baseline	SRS and scope agreed	Done
M2 Design baseline	UML diagrams reconciled (this report)	Done
M3 Core application	Login, users, courses, sections, sessions, persistence	Done
M4 Recognition integration	Camera frame returns a student or a clear reason	Done
M5 End-to-end prototype	Recognised students recorded once per session	Done

Milestone	Deliverable / exit condition	Status
M6 Verification and report	94 automated tests, recognition evaluation, demo recording, this report	Done

7 Testing Strategy and Techniques

Testing objectives: verify every functional requirement on valid and invalid paths; check that attendance is attached to the correct student, session and section; verify that repeated recognition never creates duplicates; check that failures are visible and never reported as success; and measure recognition quality and speed before claiming accuracy.

Table 7. Testing techniques applied

Technique	Application in this project	Where
Unit testing	Credential checks, session state transitions, duplicate rule, percentage calculation, CSV formatting, face-service functions (decode, embedding, save/load).	test_backend, test_face_recognition
Integration testing	Face service, attendance service and database together through the HTTP API; teacher/section scoping through the data provider.	test_live_attendance_api, test_frontend
System / end-to-end testing	Student creation to persistence, enrolment, face enrolment, session, recognition, report and CSV; restart persistence.	test_e2e_integration
Negative testing	Invalid credentials, unknown faces, no face, several faces, closed session, wrong section, unassigned section, missing payload, database write failure.	All suites
Boundary testing	Unit boundaries of the 75% flag; sessions with zero held (N/A, not 100%); dark/bright/blurry and small-face frames at the quality gate.	test_backend, test_face_sface
Security testing	CSRF missing/invalid/valid, API-key scope, unauthenticated routes, role escalation (student, faculty to admin), production secret checks.	test_security, test_sections_and_roles
Regression testing	Quoted-JavaScript session configuration, no MVP badge, admin-only controls hidden for faculty.	test_frontend
Recognition evaluation	Genuine scans under degradations, impostor scans, background replacement, look-alike analysis on the enrolled demo faces.	Section 9.2
Performance testing	25 timed recognition calls (detect, align, embed, match) on CPU.	Section 9.2
Usability testing	Not performed with independent users in this version; the interface was exercised by the team during the beta test.	Section 10

7.1 Test environment and data

- Automated tests run with pytest against throw-away SQLite databases; no test touches the real database or the real reference photos.
- Plumbing tests use synthetic frames and the legacy extractor; real-photo recognition tests use the SFace pipeline and skip automatically if the models or photos are absent.
- Recognition experiments use the demo faces supplied by team members and classmates for this purpose.

8 Test Cases

All 24 test cases below were executed on 06 November 2026 by running the automated suite (python -m pytest). Each case lists the automated test(s) that implement it. Result "Pass" means the test passed in the final run.

Table 8. Test cases, mapping to automated tests, and results

ID	Scenario	Input / precondition	Expected result	Automated test(s)	Result
TC-01	Valid faculty login	Seeded faculty credentials	Dashboard shown	test_login_with_seeded_faculty; test_valid_credentials_authenticate	Pass
TC-02	Invalid login	Wrong or arbitrary credentials	Access denied, clear error	test_invalid_password_rejected; test_arbitrary_nonempty_credentials_rejected	Pass
TC-03	Register student (admin)	Valid profile with section	Saved, auto-enrolled in the section's courses	test_new_student_is_enrolled_in_their_sections_courses; test_1_student_creation_and_persistence	Pass
TC-04	Enrol face	Usable photo / frames	Embedding linked to the student; flag set	test_4_face_encoding_persistence_and_failure_honesty; test_one_photo_gives_several_samples_and_rotation_is_tolerated	Pass
TC-05	Start session (assigned section)	Faculty, course, own section	Session Active; roster = section students	test_session_roster_is_the_chosen_section_only	Pass
TC-06	Start session (unassigned section)	Faculty, other teacher's section	Refused	test_faculty_cannot_start_unassigned_section; test_admin_reassigns_section_teacher	Pass
TC-07	No face in frame	Blank / noise frame	No record; scanning continues	test_no_face_detected_in_blank_image; test_blank_and_noise_frames_never_match	Pass
TC-08	Recognised enrolled student	Degraded live-like frame	Correct student returned and recorded	test_enrolled_face_is_recognised_in_a_degraded_frame; test_valid_recognised_enrolled_student_marked_once	Pass
TC-09	Unknown face	Face not enrolled	No attendance recorded	test_unrecognised_face_cannot_create_attendance; test_unknown_face_rejection_below_threshold	Pass
TC-10	Duplicate recognition	Same student scanned repeatedly	One present record only	test_repeated_request_does_not_create_second_attendance_record	Pass
TC-11	Wrong section / not enrolled	Student of another section	Rejected with reason	test_section_mismatch_attendance_rejection; test_student_not_enrolled_cannot_be_marked	Pass
TC-12	Close session	Faculty ends session	Status Completed; new marks rejected	test_ending_session_changes_persisted_state; test_inactive_or_closed_session_rejects_attendance	Pass
TC-13	Section-aware report	Course with sessions per section	Percentages use applicable sessions; N/A if none	test_report_percentages_are_per_section; test_attendance_reports_and_percentages	Pass
TC-14	Session report	Completed session	Present and absent lists correct	test_session_report_lists_present_and_absent	Pass
TC-15	Database failure	Forced write failure	Failure surfaced, no false success	test_database_write_failure_not_reported_as_success	Pass
TC-16	Manual correction / override	Faculty edits an entry	Update persists and appears in reports	test_manual_override	Pass
TC-17	CSV export	Report with known rows	File matches report; scoped to the teacher	test_csv_export; test_7_csv_export_multi_course_and_edge_cases; test_csv_export_is_scoped_to_the_teacher	Pass
TC-18	Admin-only actions	Faculty tries to create course / student / enrol / capture face	Forbidden	test_faculty_cannot_create_course_or_student; test_admin_roster_and_face_apis_are_admin_only	Pass
TC-19	CSRF and API key	Missing / invalid token; camera key	Rejected / accepted as specified	test_csrf_missing_token_on_browser_session_rejected; test_camera_api_key_cannot_access_administrative_endpoints	Pass

ID	Scenario	Input / precondition	Expected result	Automated test(s)	Result
TC-20	Restart persistence	Restart app and reload	Data and embeddings still present	test_6_true_restart_persistence	Pass
TC-21	Background independence	Same face, three different backgrounds	Still recognised	test_background_does_not_affect_recognition	Pass
TC-22	Low-quality frame	Very dark frame	Reported as low quality or no face; no guess	test_dark_frame_is_reported_as_low_quality_not_guessed	Pass
TC-23	Two faces in frame	Two people together	Rejected: one face at a time	test_two_faces_in_one_frame_are_rejected	Pass
TC-24	Scanner script initialises	Open a live session page	Session id is a quoted JavaScript string	test_live_session_page_config_is_valid_javascript	Pass

9 Results and Evaluation

9.1 Automated test results

Table 9. Result of the final run (94 tests, 94 passed, 0 failed, about 80 seconds)

Suite	Focus	Tests	Passed
test_backend	Database, auth, models, sessions, duplicates, reports, override, CSV	8	8
test_e2e_integration	End-to-end persistence, enrolment, biometric scenarios, CSV edge cases	8	8
test_face_recognition	Decode, embedding, registration, matching, camera handling (legacy extractor)	15	15
test_face_sface	Real-photo SFace pipeline, background, rotation, quality, roster restriction	9	9
test_frontend	Roles, section scoping, sessions, reports, CSV scope, scanner regression	21	21
test_live_attendance_api	Recognition endpoint rules and failure honesty	8	8
test_sections_and_roles	Section schema, mismatch rejection, admin-only actions	4	4
test_security	Authentication, CSRF, API key, RBAC, production safeguards	21	21
Total		94	94

9.2 Recognition evaluation

Experiments used the 9 enrolled demo faces (8 in section A: Bhoomi Saraf, Aarav Sharma, Ishita Bansal, Rohan Mehta, Diya Khandelwal, Karan Choudhary, Nisha Rajput, Aditya Saxena; and Apoorva Kala in section B), one enrolment photo each, threshold 0.50, margin 0.08.

Table 10. Recognition measurements

Experiment	Procedure	Result
Genuine scans on the section-A roster	8 students x 5 degradations (webcam-size, rotated and dim, rotated and blurred, colour cast with noise, heavy JPEG).	40 / 40 correct; 0 wrong person; 0 rejected; similarity to own template 0.88 to 0.99.
Background replacement	Same face on dark, random-noise and bright-colour backgrounds.	Recognised every time; similarity 0.98 or higher.
Tilt, size, light	Rotation +/-12 degrees, 40% size, dim, bright, blur, noise.	Recognised; similarity 0.91 or higher (over-bright frame rejected by the quality gate with a message).
Persons not on the roster	Each of 9 students scanned (3 degradations) against a roster that excludes them: 27 scans.	22 correctly rejected; 5 falsely accepted. All 5 were the same pair, Ishita Bansal and Diya Khandelwal (similarity 0.57 to 0.63). The other 7 students were never accepted as someone else.
Look-alike analysis	Best similarity between different enrolled people.	Range 0.04 to 0.72. Pairs above the threshold: Aditya/Rohan 0.72, Aarav/Aditya 0.68, Karan/Rohan 0.68, Ishita/Diya 0.61.
Speed	25 timed calls of detect, quality, align, embed and match on CPU.	Mean 63 ms, median 59 ms, 95th percentile 83 ms, maximum 108 ms.

Interpretation. Three of the nine people (Bhoomi, Apoorva, Nisha) have no other enrolled face above the threshold. Six people belong to at least one look-alike pair (Aditya, Rohan, Aarav, Karan, Ishita, Diya). For people with two close look-alikes the margin rule rejects an ambiguous scan, which is why Aditya, Rohan, Aarav and Karan were never accepted as one another in the impostor test. The one pair that was accepted, Ishita and Diya, has only a single close neighbour, so a scan of one can pass as the other when she is absent from the roster. In a real session both are on the roster, so the margin rule (best must beat second best by

0.08) and the two-scan agreement rule protect against a wrong mark; the cost is that an ambiguous scan is rejected and has to be repeated. The figure of 5 false accepts in 27 impostor scans, all from one pair, is therefore a worst case.

Limits of this evaluation. There is one enrolment photo per person, and the genuine scans are degraded copies of that same photo, so they are easier than independent webcam captures. No false-rejection rate for independent live captures was measured, and the sample is small (9 people). Webcam enrolment at the real classroom setup is expected to raise genuine similarity and widen the margin for look-alikes; this is the recommended next measurement.

9.3 Defects found in the beta test and their resolution

Table 11. Defect log

ID	Defect observed	Root cause	Fix
D-01	Live scanner never started; every scan went to /sessions/undefined/recognize.	Session id written into JavaScript without quotes.	Emitted with tojson; regression test added.
D-02	Wrong students recognised (Bhoomi shown as another student); results inconsistent between scans.	General ImageNet classifier used as the face model; whole box including background compared.	YuNet + SFace pipeline, alignment, quality gate, roster-restricted matching, margin and two-scan agreement.
D-03	Students shown as enrolled without a face; duplicate templates under several keys.	Database flag instead of real embeddings; numeric aliases.	Status from the embedding registry; one key per student.
D-04	Roster and report links pointed to wrong records.	List position used as course id.	Real course and student ids everywhere.
D-05	Percentages wrong for section sessions; empty courses showed 100%.	Report ignored sections and the zero-session case.	Per-section applicable sessions; N/A when none.
D-06	Teachers saw all courses and could not start most sessions.	No teaching-assignment model.	course_sections table; scoped dashboard, courses, students, reports, CSV.
D-07	Create-course, register-student and roster editing available to faculty.	No admin-only enforcement in the UI.	Admin-only routes and controls; 403 for faculty.
D-08	Tests depended on the developer's live database; one test run overwrote reference photos.	Hard-coded ids; photo path ignored the test sandbox.	Isolated temp databases; photos stored beside the configured embeddings file.
D-09	Intermittent duplicate session id error.	Timestamp resolution on Windows.	Random suffix added to the session id.

9.4 Acceptance criteria

Criterion	Evidence	Met
Valid faculty can start and close a session for an assigned section	TC-05, TC-12	Yes
A recognised enrolled student is recorded once, with session reference and timestamp	TC-08, TC-10	Yes
Unknown or undetected faces never produce a confirmed mark	TC-07, TC-09, TC-22	Yes
Reports match independently calculated totals for the test data	TC-13, TC-14, TC-17	Yes
Camera or database failure is communicated; no false success	TC-15, TC-22	Yes
Recognition accuracy and latency reported with conditions and limits	Section 9.2	Yes (small sample; limits stated)
Faculty see only their own sections; master data is admin-only	TC-18; test_faculty_only_sees_own_courses_and_students	Yes

10 Limitations and Future Work

10.1 Limitations

- No liveness detection: a printed photo or a video of an enrolled student could be accepted. This was excluded by the synopsis and remains the main security gap.
- Look-alike students can be confused when only one of them is on the roster; the system prefers to reject ambiguous scans, which may require a re-scan.
- The recognition evaluation is small (9 people, one enrolment photo each) and does not include independent live captures.
- Usability was not tested with independent users. Effort figures are planning values, not logged hours.
- Prototype infrastructure: SQLite, Flask development server, no HTTPS, accounts created through the seed file.

10.2 Future work

- Add liveness / anti-spoofing and evaluate it against print and replay attacks.
- Measure FAR and FRR on independent webcam captures from a larger, consented group; tune the threshold per deployment.
- Audit trail for manual corrections; e-mail alerts for low attendance; student self-service view.
- Administrator screens for faculty accounts; ERP integration; mobile access to reports.

11 Conclusion

The Smart Attendance System now covers the full workflow of the synopsis: master data with a section-wise teaching model, face enrolment, section-restricted live sessions, duplicate and wrong-section protection, reports and CSV, and role-based security. The beta test exposed serious defects, chiefly a broken scanner script and an unsuitable face model; both were fixed, the face pipeline was rebuilt around a face-specific network with quality checks and multi-scan confirmation, and the final suite of 94 automated tests passes. Recognition separates the enrolled demo faces well except for a few look-alike pairs, and the report states the limits of that evidence. The remaining work is mainly more data, liveness detection and independent usability testing.

References

[1] IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

[2] ISO/IEC/IEEE 29148:2018, Systems and software engineering: Life cycle processes: Requirements engineering.

[3] I. Sommerville, Software Engineering, 10th ed., Pearson Education.

[4] R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill.

[5] Object Management Group, UML 2.5 Specification.

[6] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li, D. Wen, "SFace: Sigmoid-Constrained Hypersphere Loss for Robust Face Recognition", IEEE Transactions on Image Processing, 2021.

[7] OpenCV Zoo, YuNet face detection and SFace face recognition models, github.com/opencv/opencv_zoo.

[8] B. W. Boehm, Software Engineering Economics, Prentice Hall, 1981 (COCOMO).

[9] CSE3132 project synopsis and laboratory records (Labs 2, 3, 5), Manipal University Jaipur.

Appendix A Running and demonstrating the system

Set-up (once): pip install -r requirements.txt; python fetch_models.py; python seed_demo.py. Start: python run_frontend_dev.py and open http://127.0.0.1:5000.

Role	Username	Password	Use in the demo
Administrator	admin	admin123	Register student, create course, assign teachers, edit roster, capture face
Faculty	rajesh.kumar	faculty123	Teaches CSE3132 sections A and C: start a section A session and scan
Faculty	meenakshi.s	faculty123	Teaches CSE3132 section B: shows the wrong-section rejection